

Subconsultas: El poder de la anidación lógica en SQL

Automatización, precisión y el manejo avanzado de datos a través de consultas multinivel.



El límite de las consultas planas y el proceso manual

Imagina buscar los automóviles que tienen un precio de alquiler superior al del coche con matrícula '5031JHL'. Sin la herramienta adecuada, nos enfrentamos a un proceso ineficiente en dos pasos:

1. La búsqueda manual:

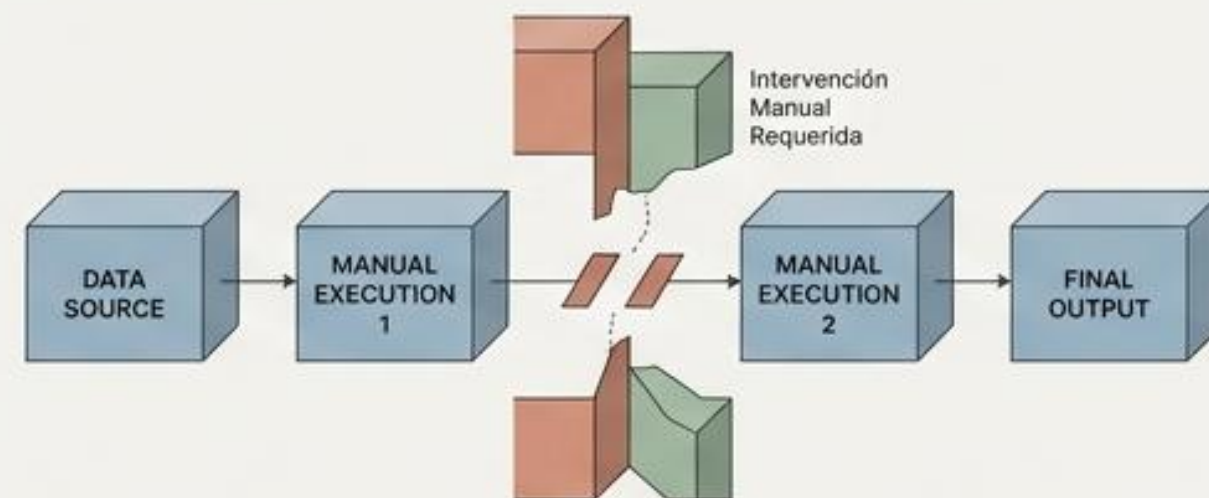
```
SELECT precio FROM automoviles  
WHERE matricula = '5031JHL';
```

(El motor nos devuelve un único dato: 116.45 Euros. Lo anotamos mentalmente o en un post-it).

2. La segunda ejecución:

```
SELECT matricula, marca, modelo, precio  
FROM automoviles  
WHERE precio > 116.45;
```

Este enfoque rompe el flujo de trabajo, impide la automatización y es propenso a errores humanos si los datos subyacentes cambian.



La revelación de la subconsulta automatiza el análisis

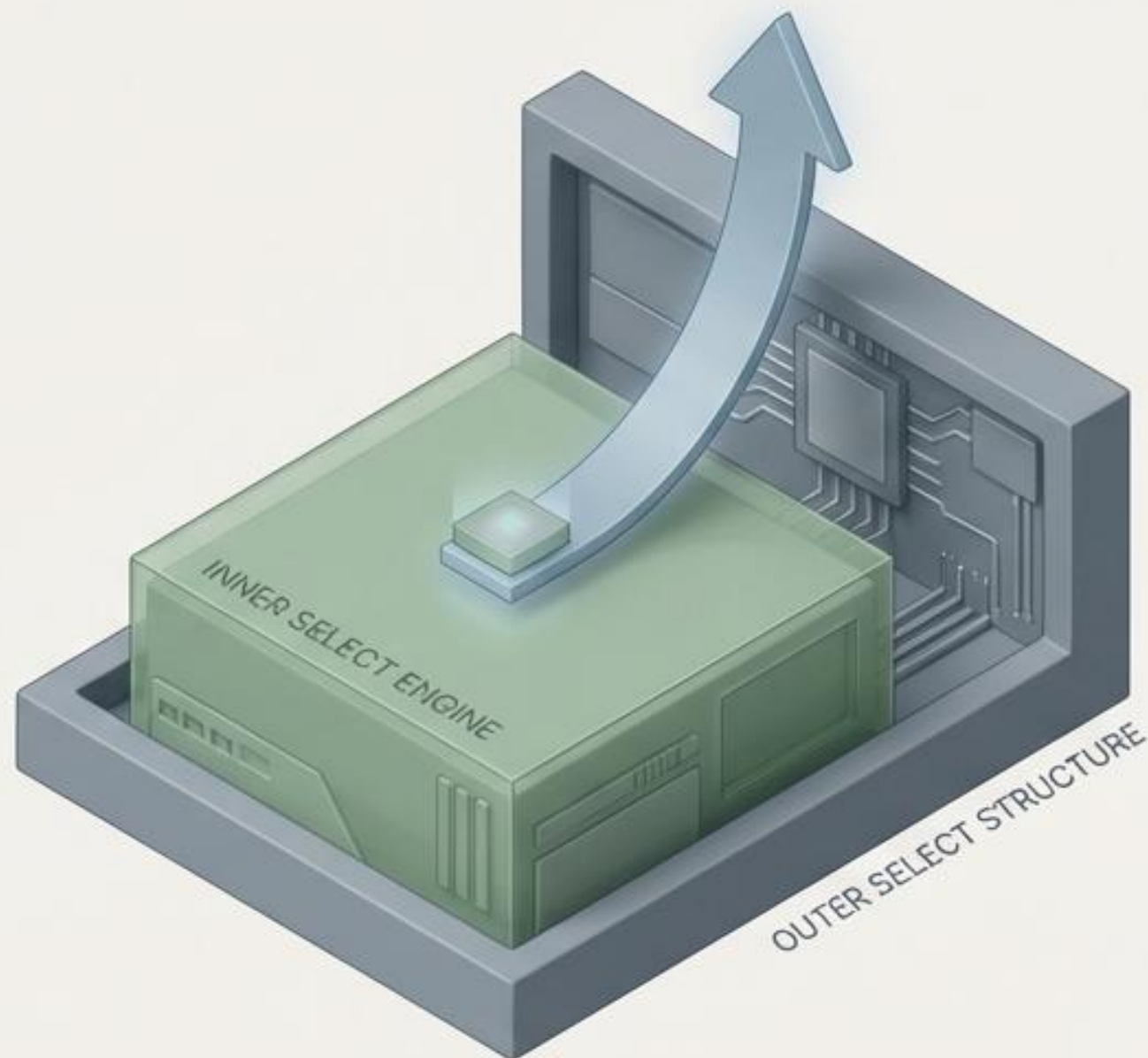
Una subconsulta es, por definición, una instrucción SELECT anidada dentro de otra instrucción. Los datos que se obtienen de esta consulta interna se inyectan dinámicamente en la consulta principal.

El fin de las consultas en dos pasos:

```
SELECT matricula, marca, modelo  
FROM automoviles  
WHERE precio > (SELECT precio FROM automoviles  
                WHERE matricula = '5031JHL');
```

Más allá de la lectura:

Aunque su hábitat natural es la instrucción SELECT, este mecanismo de anidación -de anidación empodera radicalmente otras operaciones, pudiendo utilizarse también dentro de instrucciones INSERT, UPDATE y DELETE.



La regla de oro del valor único en las comparaciones

MUY IMPORTANTE: Cuando utilizamos subconsultas para comparar con un valor mediante operadores matemáticos tradicionales ($=$, $>$, $<$, $<>$), la consulta interna actúa como un francotirador. Tiene **estrictamente prohibido devolver más de un dato**.

Si la subconsulta devuelve una lista en lugar de un único valor, el sistema colapsará al intentar evaluar la desigualdad.

Por supuesto, podemos añadir más filtros convencionales en la consulta principal para refinar el resultado. Por ejemplo, buscando solo los coches rojos que superen este umbral dinámico:

```
SELECT matricula, marca, modelo
FROM automoviles
WHERE precio > (SELECT precio FROM automoviles
  WHERE matricula = '5031JHL') AND color = 'rojo';
```



Filtrando agrupaciones enteras a través de la cláusula HAVING

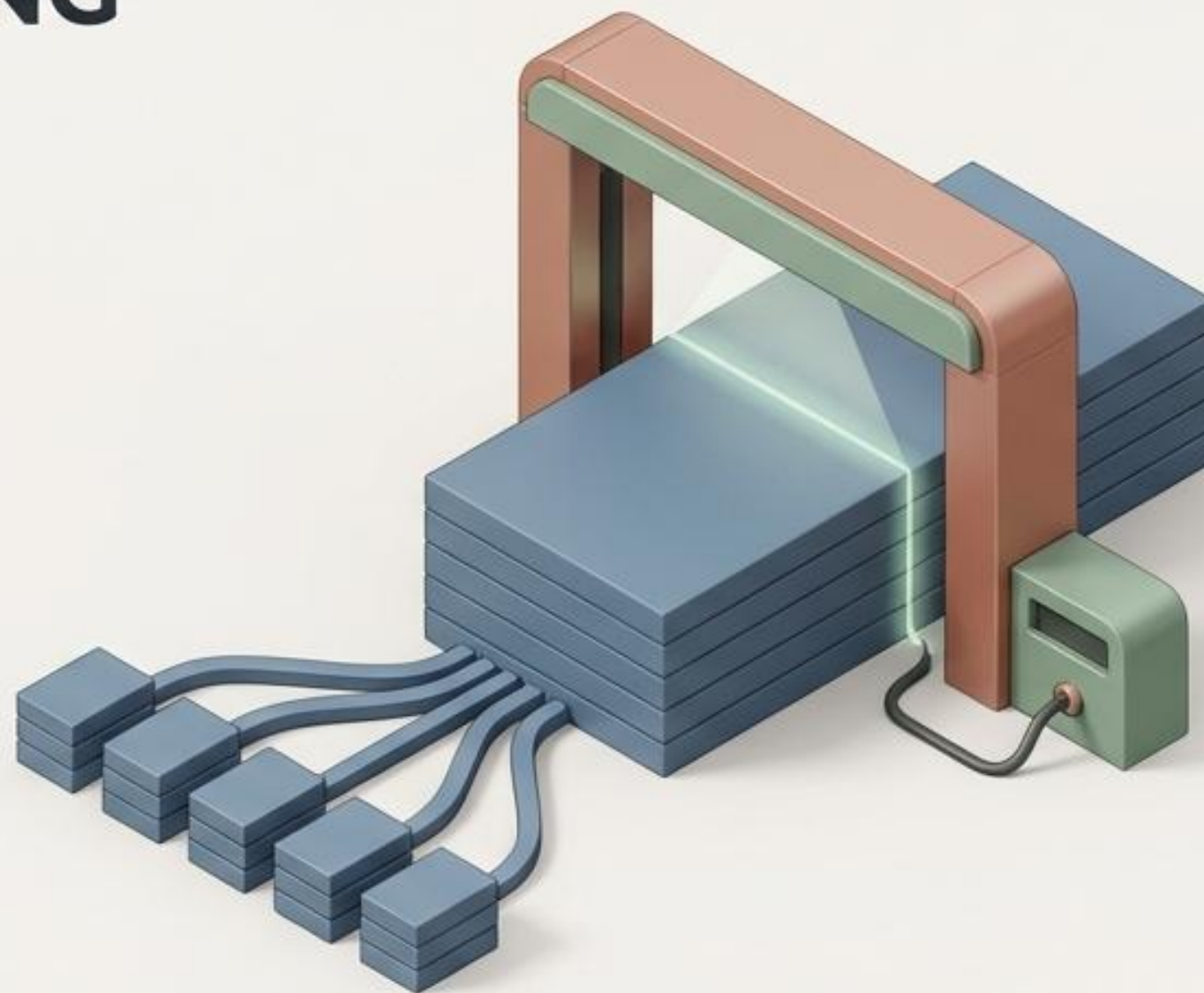
Las subconsultas no solo viven en el barrio del WHERE para evaluar registros individuales. Su flexibilidad lógica nos permite anidarlas dentro de la cláusula HAVING para condicionar resultados de funciones de agrupamiento.

El caso de uso:

Queremos obtener las marcas de vehículos y sus precios medios de alquiler, pero solo si ese precio medio es inferior al precio de un coche específico (matrícula '5031JHL').

```
SELECT marca, avg(precio)
FROM automoviles
GROUP BY marca
HAVING avg(precio) < (SELECT precio FROM automoviles
                      WHERE matricula = '5031JHL');
```

Aquí, la subconsulta proporciona el umbral de corte perfecto para evaluar métricas agregadas en tiempo real.



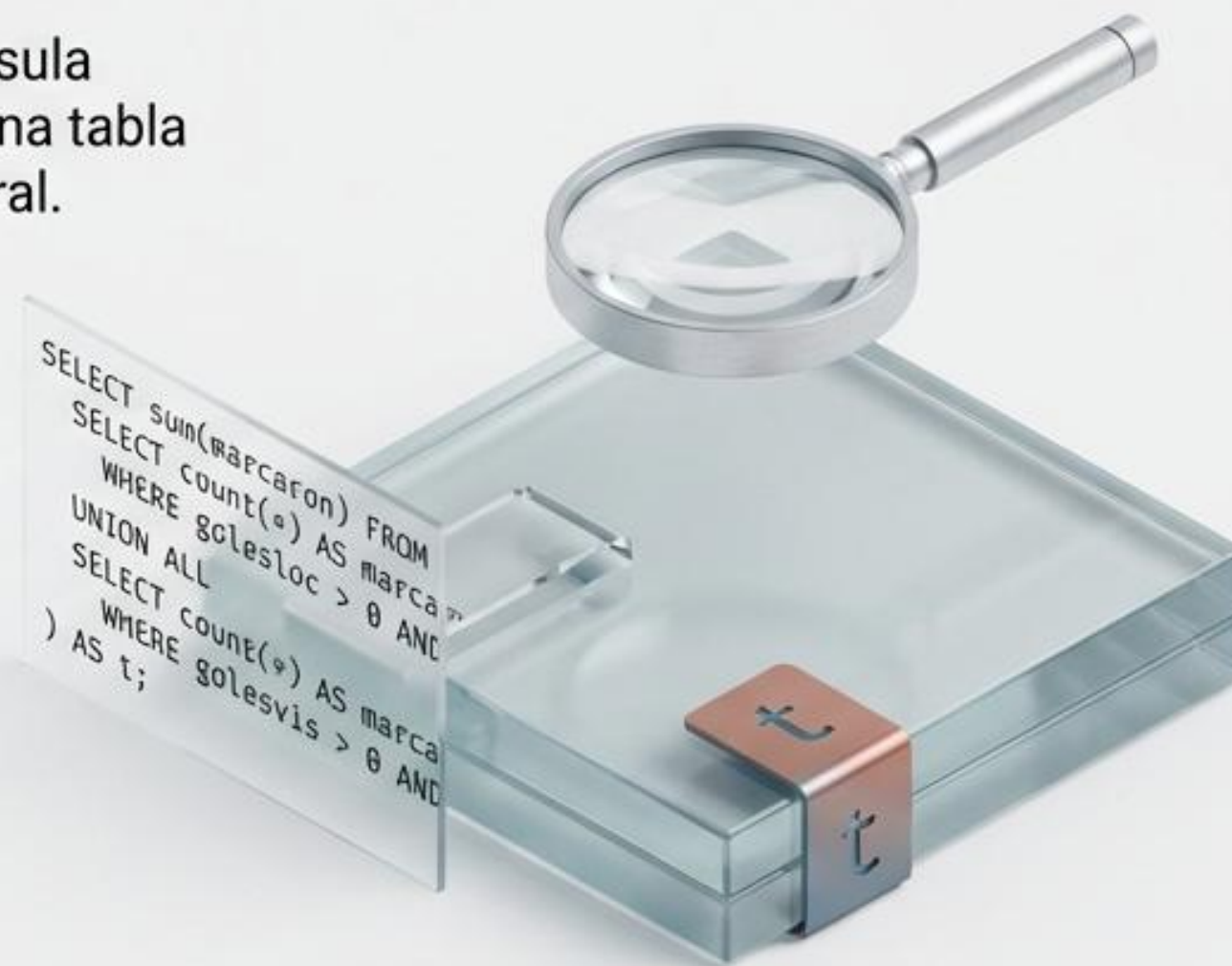
Creación de tablas virtuales al vuelo desde el FROM

El verdadero truco de magia de SQL ocurre cuando anidamos en la cláusula FROM. En lugar de actuar como un filtro lógico, la subconsulta genera una tabla virtual temporal sobre la cual podemos lanzar una nueva consulta general.

Ejemplo avanzado (Base de datos ligatercera):

Queremos calcular el total absoluto de equipos que marcaron goles en la jornada 1. Creamos una unión (**UNION ALL**) de locales y visitantes, la renombramos como tabla **t**, y sumamos sus resultados:

```
SELECT sum(marcaron) FROM (  
  SELECT count(*) AS marcaron FROM partidos  
    WHERE golesloc > 0 AND numjornada = 1  
  UNION ALL  
  SELECT count(*) AS marcaron FROM partidos  
    WHERE golesvis > 0 AND numjornada = 1  
) AS t;
```



IN: El buscador de coincidencias dentro de un conjunto

Para comprobar si un valor específico está incluido dentro del conjunto de valores devueltos por la subconsulta, utilizamos el operador IN.

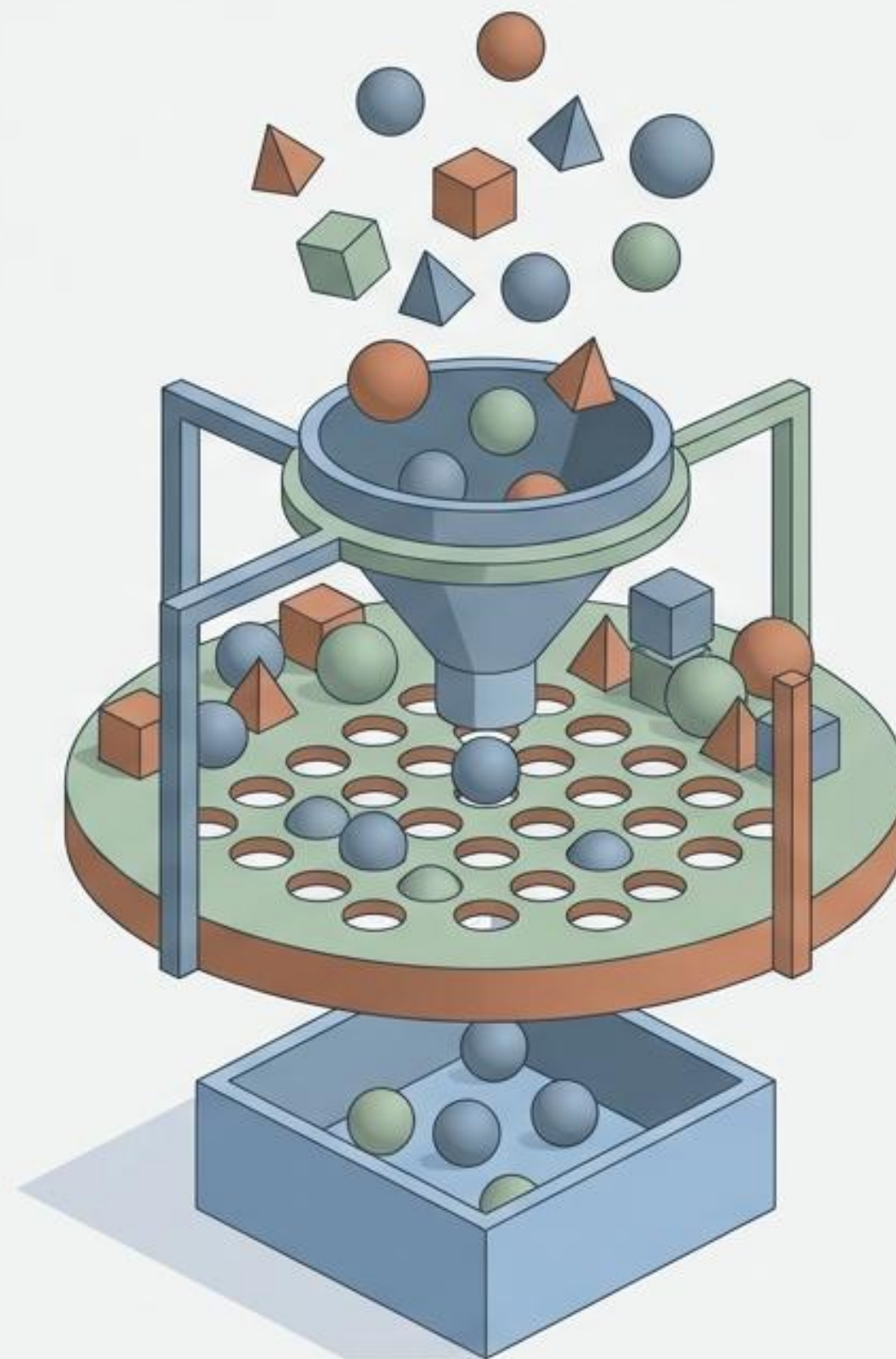
Inclusión (Coches alquilados recientemente):

```
SELECT matricula, marca, modelo  
FROM automoviles  
WHERE matricula IN  
  (SELECT matricula FROM contratos  
   WHERE fini >= '2018-01-01');
```

Exclusión (El poder de NOT IN):

También podemos invertir la lógica para encontrar ausencias absolutas. Por ejemplo, descubrir qué clientes jamás han alquilado un coche:

```
SELECT * FROM clientes  
WHERE dni NOT IN  
  (SELECT DISTINCT dnicliente FROM contratos);
```



ALL: El perfeccionista que exige cumplimiento total

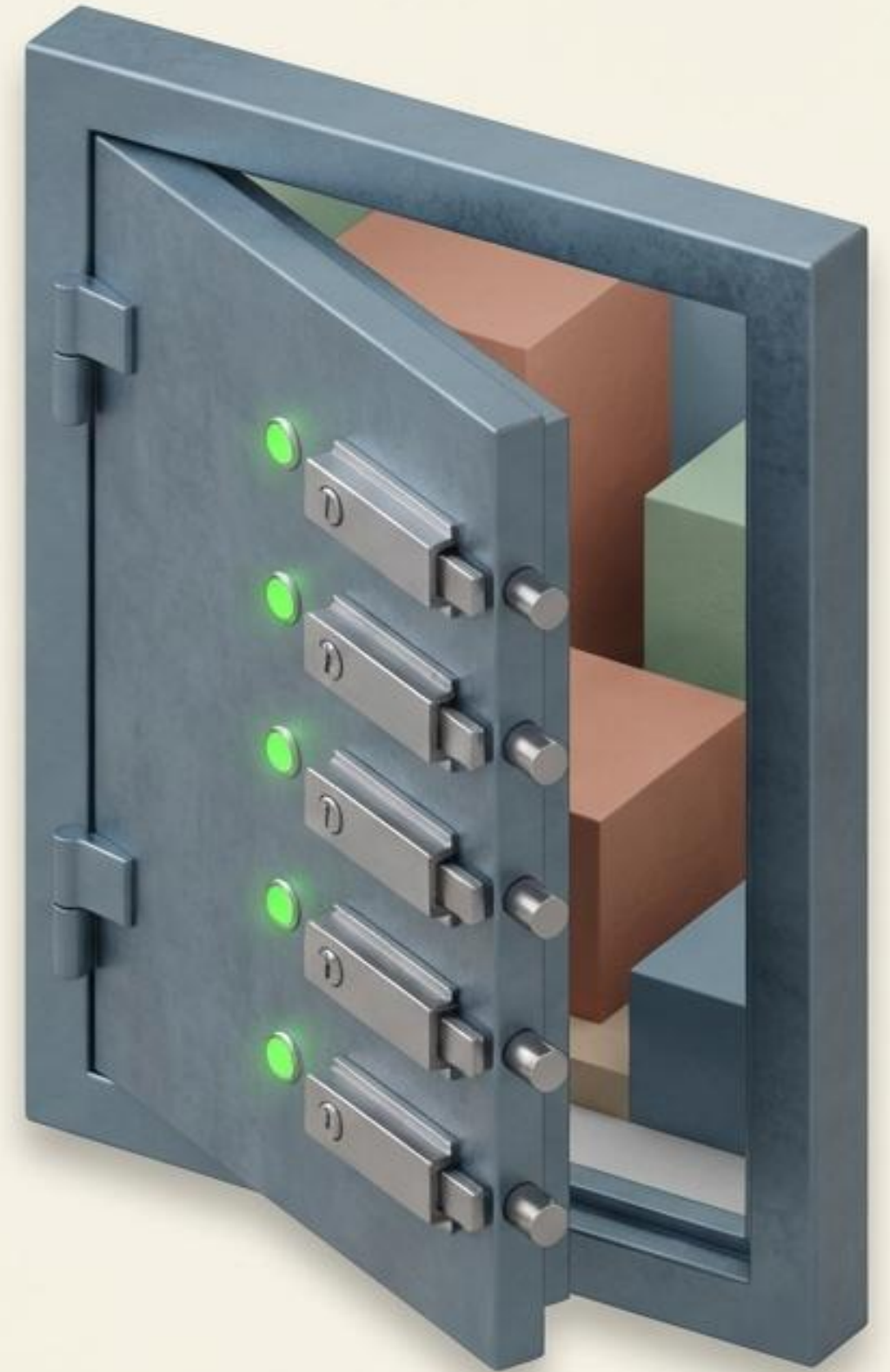
El cuantificador ALL eleva el nivel de exigencia. Permite seleccionar las filas de la consulta principal **únicamente** si cumplen la condición respecto a **todos y cada uno** de los valores devueltos por la subconsulta.

Ejemplo de aplicación:

Queremos localizar las marcas de vehículos de las que no se ha alquilado ni un solo coche durante el año 2018.

```
SELECT marca
FROM automoviles
WHERE marca <> ALL
  (SELECT DISTINCT marca FROM contratos
   INNER JOIN automoviles USING (matricula)
   WHERE year(fini) = 2018);
```

La evaluación <> ALL garantiza que la marca analizada sea diferente a la primera marca de la lista, y a la segunda, y a la tercera... sin excepciones.



ANY: El oportunista de la condición mínima viable

A diferencia de la rigidez extrema de ALL, el cuantificador ANY es altamente flexible. Selecciona las filas de la consulta principal si la condición se cumple para al menos uno de los valores del conjunto resultante.

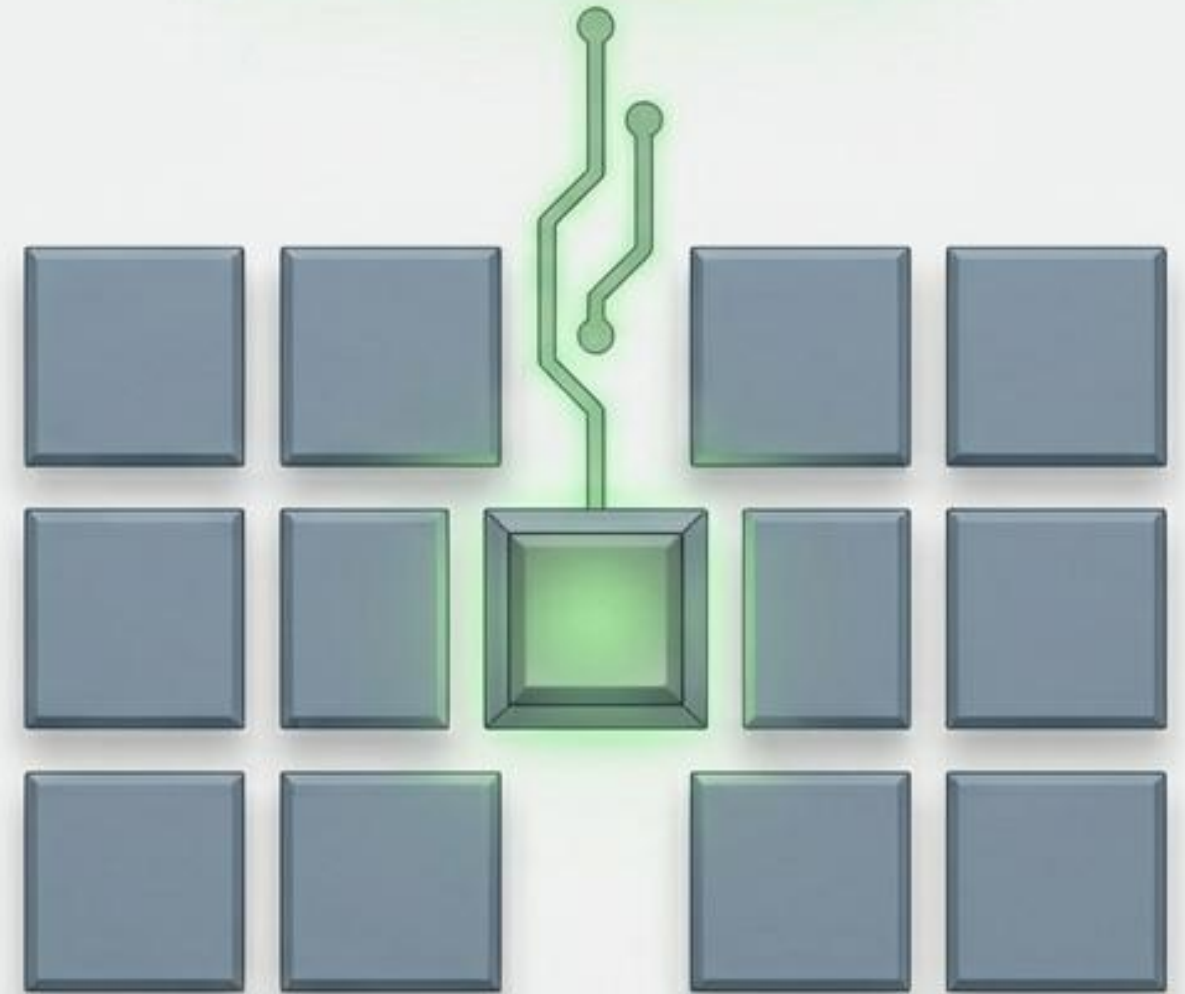
Ejemplo de aplicación:

Queremos encontrar cualquier automóvil cuyo precio de alquiler sea menor que el de algún coche de la marca SEAT (es decir, nos basta con que sea más barato que el SEAT más caro).

```
SELECT *  
FROM automoviles  
WHERE precio < ANY  
  (SELECT precio FROM automoviles  
   WHERE marca = 'seat');
```

Con que la comparación devuelva TRUE en un solo choque contra la lista, el registro es aceptado en el resultado final.

PASS



El costo oculto: Flexibilidad lógica frente a rendimiento

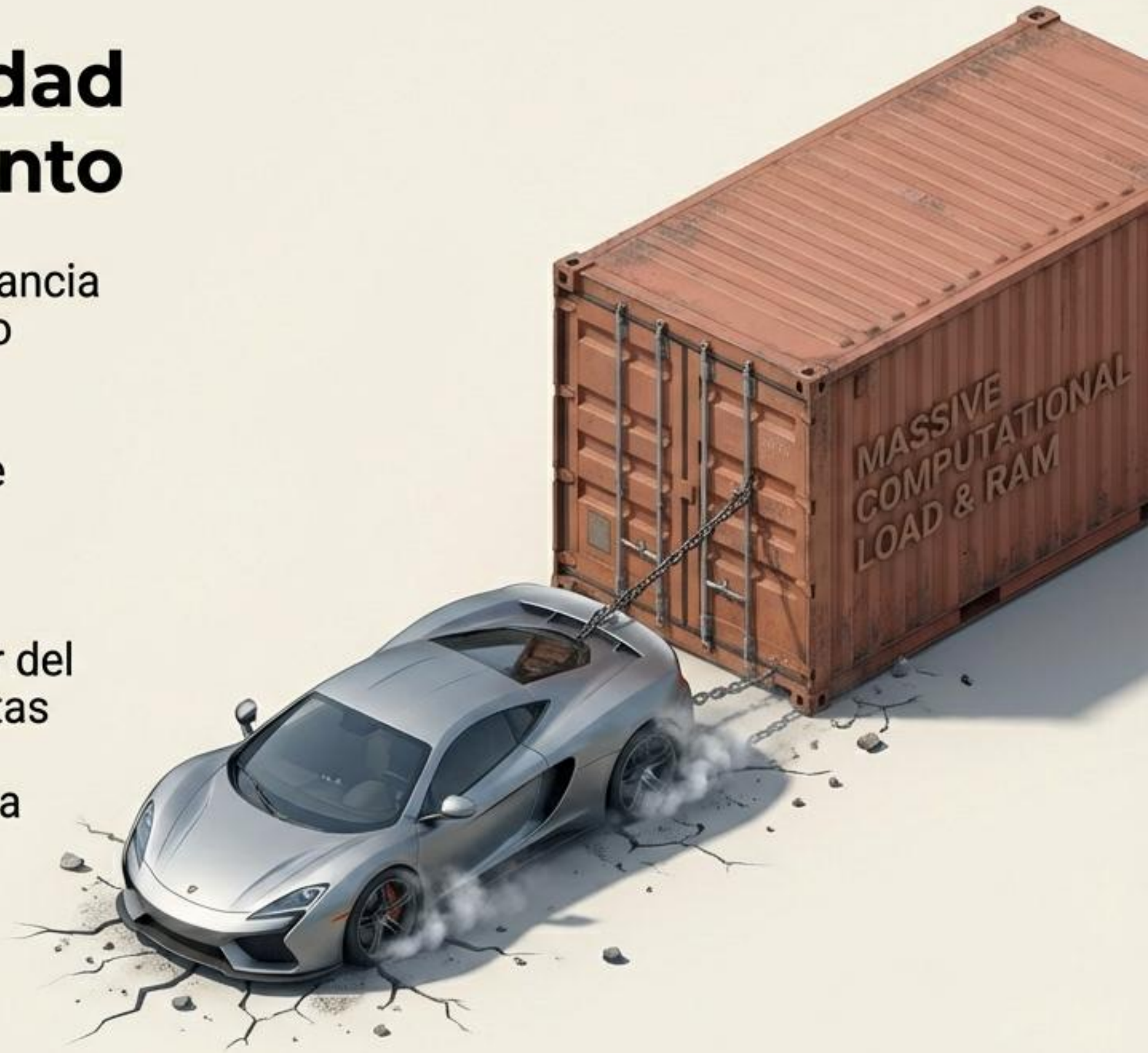
Las subconsultas son herramientas de una elegancia innegable. Nos permiten traducir el pensamiento lógico humano casi literalmente a código SQL.

Sin embargo, en bases de datos con millones de registros, esta elegancia tiene un lado oscuro.

El impacto en el motor de base de datos:

En muchos casos (dependiendo del optimizador del SGBD), las instrucciones que utilizan subconsultas consumen más tiempo de ejecución y memoria RAM. El motor puede verse obligado a ejecutar la consulta interna repetidamente por cada fila evaluada en la consulta principal.

Para cruzar datos entre entidades, existe una herramienta nativa mucho más rápida.



El duelo de arquitecturas: Subconsultas vs. JOINS

El objetivo: Obtener la marca y el modelo del coche correspondiente al contrato número 10.

El Método de la Subconsulta

**El Método de la Subconsulta
(Elegante pero pesado):**

```
SELECT marca, modelo  
FROM automoviles  
WHERE matricula =  
  (SELECT matricula  
   FROM contratos  
   WHERE numcontrato = 10);
```



El Método JOIN

**El Método JOIN (Nativo, estructural
y drásticamente más rápido):**

```
SELECT marca, modelo  
FROM automoviles  
INNER JOIN contratos  
  USING (matricula)  
WHERE numcontrato = 10;
```



Ambas consultas arrojan el mismo resultado exacto. Pero la reunión interna (INNER JOIN) es más adecuada. Las bases de datos relacionales están hiper-optimizadas en su núcleo para enlazar tablas a través de estas claves, consumiendo considerablemente menos tiempo.